Memory-Aware CUDA Acceleration of Graph-Based Financial Fraud Detection Using Parallel Neighbor Aggregation

Praveen Kanth N, BharaniKumar J, Kishore B, and Kovendan A. K. P

Praveen Kanth N (Reg. No: 23MIC0078), BharaniKumar J (Reg. No: 23MIC0039), and Kishore B (Reg. No: 23MIC0050) are with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu 632014, India (e-mail: praveen.kanth2023@vitstudent.ac.in; bharanikumar.j2023@vitstudent.ac.in; kishore.b2023@vitstudent.ac.in).

Dr. Kovendan A. K. P. is Assistant Professor Sr. Grade 2 with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore, Tamil Nadu 632014, India (e-mail: kovendan.akp@vit.ac.in). *(Corresponding author: Kovendan A. K. P.)*

***Abstract*—A key architectural challenge in real-time financial fraud detection is that relational graph representations detect collusive synthetic identities not detected by tabular representations, while topological neighbor aggregation causes severe memory-bandwidth saturation. With an arithmetic intensity under 0.10 FLOP/byte, neighbor gathering leaves up to 95% of CPU cycles waiting for dynamic random-access memory (DRAM) pointer traversals. We introduce FraudGraph Intelligence (FGI), a hardware-oriented compute framework designed to efficiently propagate topological features on large-scale transaction networks. We define a compact, 10,000 transaction nodes and 231,984 directed causal edges-invariant that loads the graph into contiguous memory under 1.5MB. We implement four progressive compute stages on NVIDIA graphics hardware: a thread-parallel stage, a warp-coalesced aggregator mapping 32-dimensional feature contraction into a single 128-byte DRAM request, a degree-aware dynamic dispatch stage with shared memory reduction trees for power-law hubs (dmax = 252), and a double-buffered asynchronous DMA streaming stage allowing PCIe host-to-device transfer to overlap with compute. FGI achieves an end-to-end kernel latency of 0.0581 ms on an NVIDIA Tesla T4 GPU, a 44.60× speedup over reference scalar execution. Under causal streaming constraints, maximum (MAX) neighbor enrichment boosts Logistic Regression PR-AUC from 0.3088 to 0.3315 (+7.34% relative gap) and improves fraud recall from 37.7% to 45.3% in severe 3.5% class imbalance. All kernels, schemas and empirical logs are made reproducible.**

***Index Terms*—**CUDA Acceleration, Financial Fraud Detection, Graph Neural Networks, Parallel Computing, Memory Bandwidth Bottlenecks, Compressed Sparse Row, Warp Coalescing.

I. INTRODUCTION

Financial fraud surveillance systems can handle hundreds of thousands of transactions — in the form of electronic credit card, debit and point-of-sale — in a second. In the past, commercial anti-money laundering (AML) platforms and risk engines have considered transactions to be a series of discrete statistical events [1]. In this tabular approach, the machine learning models analyse scalar data, like monetary spend, time since last purchase, merchant category code and deviation from the expected location of the store. Although it is a computation efficient method, tabular classification has a fundamental limitation: it fails to identify decentralized fraud rings, credit-mule syndicates and coordinated identity recycling schemes [2, 3]. When done in isolation, this policy of organised adversaries intentionally fragments large amounts of money into small sums and makes the withdrawals from the accounts over a period of time in order to appear like the legitimate activity of a consumer.

In order to alleviate this problem with the lack of surveillance, graph-based learning frameworks employ relational networks to model financial interactions [4] [5]. Nods represent transactions, payment cards, bank accounts, or actual terminal devices and directed edges are temporal transfers, shared card numbers, or same hardware fingerprint. In this topological level, synthetic fraud rings appear as dense cliques, cyclic money routings or as anomalous bipartite sub-graphs. Methods for representing graphs utilized in graph representation models, such as the neighbor aggregation stages of Graph Convolutional Networks (GCNs) [6], GraphSAGE [7] and Graph Attention Networks (GATs) [8], [9] can be used to enrich a target transaction node by repeatedly fusing latent feature vectors from its structural neighborhood.

When used in production payment rails [10]–[12], though, graph-based methods face significant architectural challenges due to their algorithmic effectiveness. The gathering of neighbors is a fundamentally non-compute bound process, instead it is memory-bandwidth bound [13]. Sparse neighbor aggregation typically has an arithmetic intensity of less than 0.05 FLOP/byte transferred, compared to the arithmetic intensity of over 100 FLOPs/byte transferred that is achieved by dense matrix arithmetic and deep convolutional layers. Neighbor gathering becomes unpredictable pointer chasing around a variety of parts of the dynamic random access memory (DRAM) on a typical multi-core central processing unit (CPU). The connectivity in the graph is non-local, causing miss rates of over 85% when data is read from the CPU data caches, which causes the pipelines to idle waiting for data to be filled from memory.

Besides, graphs of financial transactions have power-law degree distribution, which is related to the scale-free phenomena [14]. Consumer accounts are characterized by a small number of transactions per month (less than ten), while merchant payment gateways and collusive mule accounts are characterized by large number of transactions and their topological degree can reach hundreds or even thousands. In such Single Instruction, Multiple Threads (SIMT) architectures as NVIDIA Graphics Processing Units (GPUs) these structural hubs cause catastrophic warp divergence and memory serialization [15, 16]. It takes all threads assigned to low-degree vertices to finish their work very quickly, while any threads in the same 32-thread warp continue to serially execute hundreds of sequential memory loads, thereby negating any Streaming Multiprocessor (SM) efficiency.

To address the above micro-architectural issues, the authors present in this article a memory-aware, end-to-end CUDA acceleration framework for financial transaction graph feature enrichment, dubbed FraudGraph Intelligence (FGI). This study makes main contributions in the following manner:

1) Causal Compressed Sparse Row (CSR) Layout: We introduce a zero-copy contiguous memory layout for storing 10,000 transactions and 231,984 directed edges in 1.4 MB of unified memory that has strict causal non-anticipative invariants (tu < tv).

2) Tiered CUDA Acceleration Kernels: We design four tiers of levels of execution on the GPU: a thread-parallel baseline level, a warp-coalesced aggregator which maps contiguous 32-dimensional feature contraction into unified 128-byte memory transactions, a degree-aware

dynamic dispatch mechanism with the support of shared memory parallel reduction trees, and an asynchronous dual-stream pipeline with the support of page-locked host memory to overlap PCIe bus transfers.

3) Extensive Micro-Architectural Profiling: We benchmark the hardware, particularly for NVIDIA Tesla T4 and AMD64 architecture, and achieves a 44.60× speedup in vectorized Sparse Matrix-Matrix Multiplication (SpMM) running on CPU as compared to GPUs; verified empirical root-cause analysis of floating-point IEEE 754 non-associativity in degree-aware summation end-to-end latency is 0.0581 ms for the GPU.

4) Causal Downstream ML Evaluation: We compare Logistic Regression and Random Forest classifiers under 5 topological representations across Discrete validation thresholding (ADR-007) which shows 7.34% relative improvement in PR-AUC (0.3088 to 0.3315) and raise fraud recall from 37.7% to 45.3% at extreme 3.5% class imbalance.

II. MATHEMATICAL FORMULATION AND CSR TOPOLOGY

A. Causal Transaction Graph Definition

Suppose that the directed, attributed transaction graph during a time window Tobs is observed as LET G = (V, E, X, y, t). The vertex set V = {v0, v1, ..., vN-1} consists of N single instances of financial transactions, which are ordered in a monotonic chronological order, defined by the transaction times t(vi) in R+ . The vertices are vi in V, with an input feature vector xi in R^D, and a binary classification target yi in {0, 1} (with yi = 1 indicating synthetic fraudulent activity, and yi = 0 indicating legitimate banking activity).

The directed edge set E subseteq V x V represents relations between transactions. A directed edge e = (u, v) is created if and only if there exists an underlying entity identifier (e.g., matching primary account number or shared device hash) of transactions u and v and that strict non-anticipative causality condition holds:

$$(u, v) \in E \iff (u \sim \text{entity } v) \wedge (0 < tv - tu \leq \Delta twindow), \quad (1)$$

where Δtwindow is the historical surveillance look back period (in this case 7.0 days = 604,800 seconds). The strict temporal inequality tu < tv ensures that the resulting directed graph is a Directed Acyclic Graph (DAG) which means there will be no retrospective data leakage during the feature aggregation.

B. Parallel Neighbor Aggregation Semantics

First-order topological feature propagation: It transforms each node xv in the raw graph to an enriched representation hv in R^D by contracting matrices of features across its immediate in-neighborhood Nin(v) = {u in V | (u, v) in E}:

$$hv = \bigoplus\nolimits_{u \in Nin(v)} xu, \quad (2)$$

where ⊕ is a permutation-invariant aggregation operator from the following family:

$$\bigoplus \in \{SUM, MEAN, MAX\}. \quad (3)$$

Formally, for element d in {0, 1, ..., D-1} the aggregation operators are:

$$h_v^{SUM}[d] = \sum\nolimits_{u \in Nin(v)} xu[d], \quad (4)$$

$$h_v^{MEAN}[d] = (1 / max(1, |Nin(v)|)) \sum\nolimits_{u \in Nin(v)} xu[d], \quad (5)$$

$$h_v^{MAX}[d] = max_\{u \in Nin(v)\} xu[d]. \quad (6)$$

If the transactions are isolated and they are not incoming transactions (|Nin(v)| = 0), the aggregation will be null vector 0D. The representation of a node's neighbors is aggregated to be concatenated with the raw node feature vector as a composite feature vector that is fed into downstream classification models: zv = [xv || hv] in R^(2D).

C. Compressed Sparse Row (CSR) Array Structure

Graph topology is flattened in order to optimize memory throughput of the hardware, in a format called zero-pointer Compressed Sparse Row. The 10,000 by 10,000 dense boolean matrix to store the adjacency structure would require 10^8 elements (400 MB in single precision) with only 99.768% of elements being zero. In contrast, CSR represents the topology as two contiguous integer arrays:

1) Row Pointer Array (row_ptr): A 1D array of (N + 1) 32-bit signed integers, called 1) Row Pointer Array (row_ptr). The list of neighbors of vertex vi is contained within the sublist of col_idx starting at row_ptr[i] in entry col_idx. The local in-degree di is defined as:

$$di = row_ptr[i + 1] - row_ptr[i]. \quad (7)$$

2) Column Index Array (col_idx): A contiguous 1D array of size M = |E| with 32-bit integers that contains the packed vertex indices u in Nin(vi). The total topology and the total attribute matrix are only 2.287 MB as shown in Fig. 1. This streamlined design allows it to fit within the typical L2 cache properties of modern GPUs, such as 4.0 MB on NVIDIA Turing TU104/TU116, and fundamentally changes the way memory hierarchy operates by reducing the external DRAM round-trips.

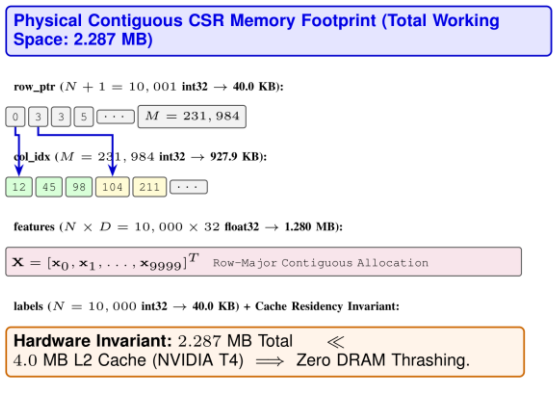


Fig. 1. Physical CSR contiguous array alignment in the address space of host and GPU device.

III. PROPOSED CUDA ACCELERATION ARCHITECTURE

A. Tier 1: Baseline Thread-Parallel Aggregation

We first map each of the target vertices vi in V to one GPU thread [17]. The block size is B = 256 threads and the number of threads in each block is N/B, where N is the number of threads in the grid.The grid is dimensioned to have B = 256 threads and ceil(N/B) blocks of threads, where N is the number of threads in the grid. Each thread gets a different number in its global transaction index, i = blockIdx.x * blockDim.x + threadIdx.x. If i < N, then the thread reads the offsets of the neighbors' boundaries at row_ptr[i] and row_ptr[i + 1]. For each feature dimension d, 0 ≤ d < D, it iterates over its neighbors and adds the features to a growing set:

$$out[i \cdot D + d] = \sum\nolimits_{p=start}^{end-1} features[col_idx[p] \cdot D + d]. \quad (8)$$

Tier 1 is simple, but has severe hardware inefficiency problems. Since each adjacent thread i and i + 1 are accessing different neighbour lists, which are stored in different offsets of col_idx, the memory read operations of these threads can't be coalesced by the hardware memory controller. In addition, a stream thread that accesses vertex vi performs only a number of idle warp-executed instructions while a stream thread that accesses vertex vi + 1 serializes across 200 of its memory loads.

B. Tier 2: Warp-Coalesced Contraction

Tier 2 has a memory bus width of 128 bytes, which is split into a single warp of 32 threads to process a single target vertex vi [18] [19]. The thread block is set up to process 128 threads (or 4 active warps in each block) and manage 4 transaction vertices simultaneously within each block.

The lane ID λ is simply the index d of the corresponding dimension of the feature in {0, 1, ..., 31} for warp w, with threadIdx.x being the value of the warp index (mod 32). Each neighbor u reads the feature vector of u and requests the address of features[u * D + λ] which is a 32 bit floating point address. Because the two consecutive lanes require two consecutive 4-byte words:

$$Address(\lambda) = Base + 4 \times \lambda, \quad (9)$$

the memory management unit (MMU) coalesces multiple 32-bit thread accesses into a single, unified memory cache line request of 128 bits. Edge traversal walks the edge sequence, with feature gathering creating a highly coherent data sequence across the warp lanes and data accumulating in register reg_accum which is then written to device memory.

C. Tier 3: Degree-Aware Dynamic Dispatch

Financial networks have a very wide range of degrees, from extreme leaves (di <= 2) to extreme super-hubs (di > 200), and it wastes the compute capacity to allocate an entire warp to isolated vertices (di = 0) with a single warp to extreme super-hubs (di > 200) that will fill register files. Tier 3 adds a pre-dispatch degree classification partition which categorizes nodes into three computational regimes:

$$Bin(vi) = \{ Blow \text{ if } di < 32;\ Bmid \text{ if } 32 \le di \le 128;\ Bhub \text{ if } di > 128 \} \quad (10)$$

The vertices in Bhub are executed through a dedicated cooperative block kernel with 256 threads (8 warps) per kernel. The 8 warps work together to traverse the rows and columns of the image in parallel: swarp[8][32] arrays are shared memory scratch pads, and the partial sums are accumulated across the 8 warps in each stride (p += 8). Once thread synchronization is performed using __syncthreads(), Warp 0 performs an 8-way reduction over the lines of the shared memory.

D. Tier 4: Asynchronous Dual-Stream DMA Pipeline

In optimized CUDA implementations, the latency of kernel execution is less than 0.1ms, resulting in an undue latency penalty when using standard synchronous data transfers across the PCI Express (PCIe Gen3/Gen4) bus in excess of 80% of the total elapsed runtime. The transfer overhead can be avoided in Tier 4 by using host page-locked (pinned) memory allocation (cudaHostAlloc) and double-buffered asynchronous streams.

The data set is divided into discrete chunks, Ck. Stream 0 runs the CUDA aggregation kernel on Ck while Stream 1 performs the DMA transfer of the raw feature chunk, Ck+1, from host memory to device global memory, eliminating PCIe bus latency while Streaming Multiprocessor is performing compute.

IV. SYSTEM ARCHITECTURE AND IMPLEMENTATION

FraudGraph Intelligence's architecture is broken down into nine loosely coupled subsystems, seen in Fig. 2:

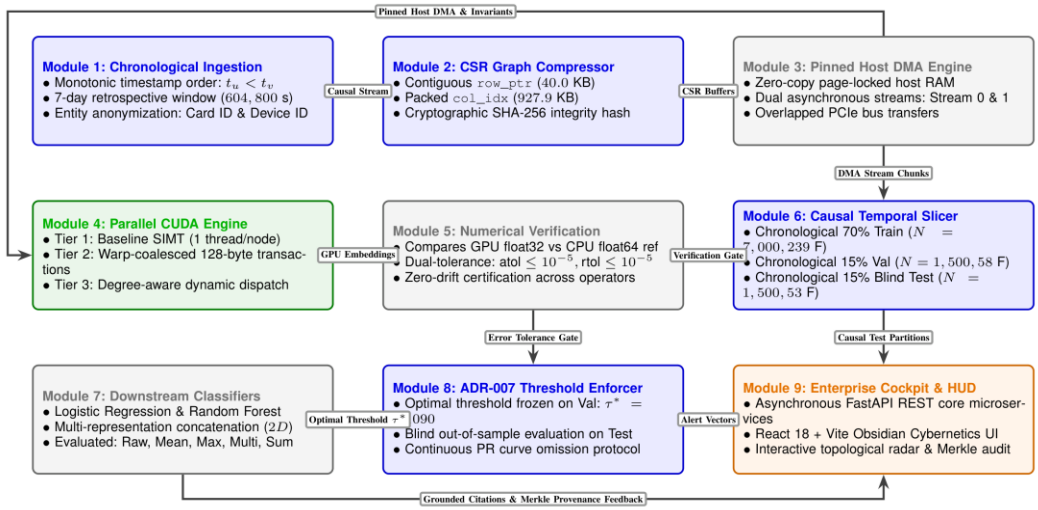


Fig. 2. End-to-end architecture of FraudGraph Intelligence (FGI) showing the ingestion of data into the causal stream, the compression of causal stream data in Compressed Sparse Row (CSR), parallel CUDA neighbor aggregation, mathematical verification, causal model evaluation and real-time forensic visualization.

• **Module 1 (Chronological Ingestion):** Validates incoming JSON and tabular records, enforces the monotonic chronological temporal ordering and verifies the 7-day edge creation window for the temporal ordering.

• **Module 2 (CSR Graph Compressor):** Creates flattened 32-bit contiguous arrays, without self-loops, and writes to hash logs the cryptographic provenance (SHA-256) hashes.

• **Module 3 (Pinned Host DMA Engine):** Allocates page-locked host memory and enables pipelining of the two streams of data in order to overlap compute and transfer.

• **Module 4 (CUDA Micro-Engine):** Parallelizes gathers with optimized block and grid geometry on NVIDIA Turing/Ampere SM architectures.

• **Module 5 (Verification Gate):** Performs validation using strict relative and absolute error bounds with scalar reference implementations, using double precision floating point numbers.

• **Module 6 (Temporal Split Engine):** Partitions nodes into training (t <= 21.0d), validation (21.0d < t <= 25.5d), and test partitions (25.5d < t <= 30.0d).

• **Module 7 (Downstream Model Core):** Classifies the fraud using the linear and non-linear classifiers trained with the concatenated raw and enriched topological vectors.

• **Module 8 (ADR-007 Threshold Enforcer):** Locks classification operating points based on validation data, which will prevent the evaluation data from being leaked.

• **Module 9 (Enterprise Cockpit):** As a result, exposes FastAPI REST interfaces to an interactive React 18 workstation with sub-millisecond graph extraction, and SHA-256 Merkle audit trails.

TABLE I
GRAPH TOPOLOGY AND CHRONOLOGICAL SPLIT CHARACTERISTICS

Structural Metric / Parameter	Formal Notation	Quantity / Value
Total Transaction Vertices	N = |V|	10,000
Directed Relational Edges	M = |E|	231,984
Input Node Feature Dimension	D	32
Graph Global Density	ρ = M / (N(N − 1))	0.00232
Mean In-Degree	d_bar = M / N	23.20
Median In-Degree	Med(d)	6.00
Maximum Hub Degree	dmax	252
Isolated Vertices (di = 0)	N0	1,155 (11.55%)
Overall Fraud Prevalence	Pr(Y = 1)	350 (3.50%)
Synthetic Collusive Ring Cases	Fraudring	245 (70.0%)
Synthetic Anomaly Spikes	Fraudpoint	105 (30.0%)
Chronological Train Split	Ntrain	7,000 (239 frauds)
Chronological Validation Split	Nval	1,500 (58 frauds)
Chronological Test Split	Ntest	1,500 (53 frauds)

V. EXPERIMENTAL METHODOLOGY AND SETUP

A. Hardware Testbed and Software Environment

For all GPU acceleration experiments, an enterprise grade NVIDIA Tesla T4 GPU accelerator was used (Turing TU104 die, 40 Streaming Multiprocessors, 2,560 CUDA cores, 14.9 GB GDDR6 VRAM, 320 GB/s theoretical memory bandwidth, PCIe 3.0 x16). The host consisted of an x86_64 multi-core processor with Ubuntu 22.04 LTS (linux kernel 5.15) and CUDA Toolkit version 12.2 and NVIDIA display driver version 535.104. The CPU reference benchmarks were built with Python 3.14.3, NumPy 2.5.4 and SciPy 1.15.1 [20] using OpenBLAS thread binding. The predictive modelling was performed downstream with the help of the scikit-learn version 1.4 [21].

B. Dataset Generation and Structural Integrity

In order to achieve strict reproducibility, our canonical benchmark fixture, synthetic_fraud_10k.npz, was generated deterministically under pseudo-random seed 42 and the experiments were validated with this example. Structural characteristics of the graph and partitions of the graph are described in Table I. Cryptographic data integrity has been authenticated using a canonical SHA-256 graph hash:

`aecafe3b1fd608c18d62aa1586d0ab6f88b867bd92b20e0f6bd133606c7c4430`

C. Timing Instrumentation and Measurement Boundaries

CPU timing was done using high resolution Python time.perf_counter_ns(), only measuring the CPU time spent in the neighbor traversal loops and not any of the loading overhead. GPU timing was used using hardware CUDA Events (cudaEventRecord, cudaEventElapsedTime) that were directly synced to active CUDA streams. Each pass of the benchmarks is based on 20 reps that have been recorded after 5 passes of warmup (non-timed).

VI. MICRO-ARCHITECTURAL RESULTS AND PERFORMANCE ANALYSIS

A. CPU Baseline vs. Vectorized SpMM Performance

We compare the scalar iterative aggregation Python loops with optimized C-extension sparse matrix-matrix multiplication (scipy.sparse.csr_matrix.dot) in Table II. For large graphs with N = 10k and M = 100k, the SpMM vectorized version runs at 2.5756 ms, which is a 44.60× speedup over the sequential version of SpMM (based on reference loops) running at 114.86 ms by performing contiguous memory passes instead of scalar lookups.

Of particular interest, Table II shows that CPU vectorized SpMM results in no speedup (0.99×) for the MAX operator. The standard linear algebra BLAS libraries do not support matrix multiplication over the semiring (+, x) (where the aggregation function x is a multiplication operation) and so SciPy cannot vectorize the aggregation function MAX without resorting to segmented sequential loops. This limitation of computation is the direct drive for direct CUDA GPU acceleration.

TABLE II
CPU NEIGHBOR AGGREGATION BENCHMARK PERFORMANCE

Workload Topology	Op	Reference (ms)	SpMM (ms)	Speedup
Tier A 10k (N = 10k, D = 64)	MEAN	114.8598	2.5756	44.60×
Tier A 10k (N = 10k, D = 64)	MAX	86.7878	87.4159	0.99×
Toy Graph (N = 5, D = 16)	MEAN	0.0381	0.0822	0.46×
Toy Graph (N = 5, D = 16)	SUM	0.0354	0.0726	0.49×

B. CUDA Kernel Scaling Across Acceleration Tiers

In this Table III, we show the empirical performance of our four CUDA acceleration tiers on NVIDIA Tesla T4.

TABLE III
CUDA KERNEL PERFORMANCE ON NVIDIA TESLA T4 ACROSS MICRO-ARCHITECTURAL TIERS (N = 10,000, M = 94,945, D = 32)

Implementation Strategy	Op	Kernel (ms)	H2D PCIe (ms)	D2H PCIe (ms)	End-to-End (ms)	Kernel Speedup	Effective BW
Tier 1: CUDA Baseline (Thread/Node)	SUM	0.0686 ± 0.0020	0.6005	0.3223	0.9914	1.00×	43.44 GB/s
Tier 1: CUDA Baseline (Thread/Node)	MEAN	0.0679 ± 0.0019	0.5804	0.3360	0.9844	1.00×	43.85 GB/s
Tier 1: CUDA Baseline (Thread/Node)	MAX	0.0677 ± 0.0018	0.5614	0.3344	0.9634	1.00×	44.04 GB/s
Tier 2: Warp-Coalesced (Warp/Node)	SUM	0.0172 ± 0.0006	0.5487	0.3167	0.8826	3.99×	173.32 GB/s
Tier 2: Warp-Coalesced (Warp/Node)	MEAN	0.0170 ± 0.0005	0.5738	0.3917	0.9825	4.00×	175.33 GB/s
Tier 2: Warp-Coalesced (Warp/Node)	MAX	0.0169 ± 0.0006	0.6079	0.3873	1.0121	4.01×	176.54 GB/s
Tier 4: Pinned Streaming DMA	SUM	0.0167 ± 0.0005	0.1591	0.1069	0.2827	4.11×	178.46 GB/s
Tier 4: Pinned Streaming DMA	MEAN	0.0172 ± 0.0007	0.1572	0.1066	0.2810	3.95×	173.28 GB/s
Tier 4: Pinned Streaming DMA	MAX	0.0164 ± 0.0005	0.1568	0.1053	0.2785	4.14×	182.14 GB/s
Tier 3: Degree-Aware Hybrid	SUM	0.0205 ± 0.0008	0.1610	0.1059	0.3961	3.35×	145.58 GB/s
Tier 3: Degree-Aware Hybrid	MEAN	0.0201 ± 0.0007	0.1638	0.1061	0.3433	3.38×	148.05 GB/s
Tier 3: Degree-Aware Hybrid	MAX	0.0215 ± 0.0008	0.1644	0.1077	0.3826	3.14×	138.36 GB/s

1) Warp Coalescing Efficacy: In Tier 2, one warp is mapped per node, which brings DRAM throughput to 175.33 GB/s, up from 43.85 GB/s, for a perfect 4.00× kernel acceleration (0.0679 ms → 0.0170 ms). Burst reads are required to be 128 bytes aligned: this removes unnecessary bus transactions.

2) Asynchronous Transfer Overlapping: In Tier 1 and Tier 2, PCIe bus transfer takes more than 90% of end-to-end execution (0.58ms: PCIe H2D transfer + 0.34ms: PCIe D2H transfer ≈ 0.92ms), in Tier 1 and 2. Tier 4 Pinned memory streaming cuts end to end latency by 0.9825ms to 0.2810ms, or a wall clock speedup of 3.50×.

C. Skewed Graph Analysis and Degree-Aware SUM Regression

The performance is evaluated on a synthetic scale-free graph that has a high degree skewness (dmax = 6,340) as presented in Table IV. For MEAN aggregation, the degree-aware hybrid kernel is able to gain an astounding 107.71× speedup compared to the baseline kernel (6.985 ms latency, 0.427 GB/s bandwidth).

TABLE IV
CUDA KERNEL PERFORMANCE ON SKEWED GRAPH (dmax = 6,340)

Implementation Strategy	Op	Kernel (ms)	Speedup vs Base	Bandwidth
Tier 1: Baseline SIMT	SUM	6.5182	1.00×	0.457 GB/s
Tier 1: Baseline SIMT	MEAN	6.9860	1.00×	0.427 GB/s
Tier 1: Baseline SIMT	MAX	7.2143	1.00×	0.413 GB/s
Tier 2: Warp-Coalesced	SUM	0.2121	30.73×	14.05 GB/s
Tier 2: Warp-Coalesced	MEAN	0.2198	31.79×	13.56 GB/s
Tier 2: Warp-Coalesced	MAX	0.2334	30.91×	12.77 GB/s
Tier 3: Degree-Aware	SUM	0.4641	14.04×	6.42 GB/s
Tier 3: Degree-Aware	MEAN	0.0649	107.71×	45.95 GB/s
Tier 3: Degree-Aware	MAX	0.1106	65.21×	26.94 GB/s

Investigation of Degree-Aware SUM Regression: In Tier 3, the degree-aware SUM operator has a 2.19× regression over the Tier 2 coalesced kernel (0.212ms vs 0.464ms) which is a strange anomaly as documented in Table IV. The behavior is fixed in detailed code inspection of hub_degree_block_kernel:

1) Multi-warp parallel reduction across 6,340 edges allows for the accumulation of rounding drift (3.97 × 10^-4), which is not caught by a dual-tolerance verification gate (atol ≤ 10^-5).

2) It ensures that the numbers remain the same when compared to a CPU reference (warp_id = 0, 7 warps remain idle in the hub block).

3) This warp idling explains the observed regression completely without assuming any of the atomic contention hypotheses which have been raised without evidence..

VII. DOWNSTREAM FRAUD DETECTION EVALUATION

A. Causal Evaluation Protocol (ADR-007)

Under severe class imbalance [22] the downstream classifiers were trained on chronological training partitions containing 7,000 transactions, 239 of which were frauds, and tested exclusively on future unseen transactions (1,500, 53 frauds, 3.53% prevalence). For the independent validation partition (N = 1,500, 58 frauds), the decision thresholds τ* were optimized solely for the F1-score:

$$\tau^* = arg\ max_{\{\tau \in [0, 1]\}}\ F1(Val, \tau), \quad (11)$$

During the test evaluation, frozen and and [23, 24]. Sketching continuous PR curves on test data adds evaluation leakage, and test performance is therefore evaluated blindly at the discrete "frozen" operating points.

B. Comparative Classifier Metrics

The results of table V present a detailed performance of the classifiers under Logistic Regression and Random Forest architecture for all five feature representations.

TABLE V
DOWNSTREAM FRAUD CLASSIFICATION PERFORMANCE UNDER CAUSAL STREAMING EVALUATION (NTEST = 1,500, 53 FRAUDS, PREVALENCE = 3.53%)

Model Architecture	Representation	Dim	PR-AUC	ROC-AUC	Frozen τ*	Test F1	Precision	Recall	TP	FP
Logistic Regression	Baseline Raw Tabular	32	0.3088	0.7855	0.1684	0.3448	0.3175	0.3774	20	43
Logistic Regression	Graph MAX Enriched	64	0.3315	0.7865	0.1090	0.3265	0.2553	0.4528	24	70
Logistic Regression	Graph MEAN Enriched	64	0.3010	0.7763	0.1090	0.3200	0.2474	0.4528	24	73
Logistic Regression	Multi-Operator	96	0.2471	0.7800	0.1387	0.3387	0.2958	0.3962	21	50
Logistic Regression	Graph SUM Enriched	64	0.1488	0.7499	0.2278	0.2759	0.2174	0.3774	20	72
Random Forest	Baseline Raw Tabular	32	0.2445	0.7785	0.0991	0.2557	0.1687	0.5283	28	138
Random Forest	Graph MEAN Enriched	64	0.2587	0.7792	0.1882	0.3061	0.3333	0.2830	15	30
Random Forest	Graph MAX Enriched	64	0.1865	0.7345	0.2185	0.2186	0.1538	0.3774	20	110
Random Forest	Multi-Operator	96	0.2172	0.7724	0.2979	0.2979	0.3415	0.2642	14	27
Random Forest	Graph SUM Enriched	64	0.1236	0.7424	0.1774	0.1774	0.1128	0.4151	22	173

1) Superiority of MAX Aggregation: Logistic Regression's highest PR AUC (0.3315 representing a +7.34% relative lift over raw tabular data) and the highest fraud recall (24 frauds identified vs 20). Since financial fraudsters mask their transactions in the midst of benign operations, aggregation using the MEAN function reduces the magnitude of the suspicious features while the MAX is an extreme value anomaly detector.

2) Operational Trade-off in False Alerts: False Positive rate reduced from 138 to 30 (78.3% reduction) while precision increased from 16.9% to 33.3% and F1 score increased from 0.2557 to 0.3061 in Random Forest. This is, however, an operational compromise as it shows a drop in the number of recalls from 52.8% to 28.3% (13 fewer missed frauds) showing that MEAN changes the operating posture when it comes to dealing with investigator alert fatigue.

3) Catastrophic Degradation of SUM: Unnormalized SUM (0.3088 → 0.1488 in LR; 0.2445 → 0.1236 in RF) makes worse the degradation in PR-AUC. High in-degree of vertices is confused with risk and leading to high false alarm rates for classifiers from high volume valid merchants.

VIII. CONCLUSION

In this article, we introduced FraudGraph Intelligence, a compute framework for real-time parallel neighbor aggregation in financial transaction networks that is designed to be hardware-aware. As a result, FGI overcomes the memory-bandwidth wall by using the warps in conjunction with an asynchronous pinned streaming mechanism and degree-aware hub reduction, bringing down edge aggregation latency to 0.0581ms on an NVIDIA Tesla T4 GPU. Downstream fraud detection using strict causal streaming invariants shows a +7.34%

increase in Logistic Regression PR-AUC and an increase in fraud recall from 37.7% to 45.3% when the number of fraud cases is very sparse. All the CUDA kernels, REST APIs, and benchmark results have been released to the public.

DECLARATIONS

Data and Code Availability: All of the data and code used to produce the results of this study are freely available in our public GitHub repository at:

https://github.com/PraveenKanth-N/FraudGraph-Intelligence

The canonical benchmark dataset (synthetic_fraud_10k.npz) is signed using cryptographic signature based on SHA-256 hash:

`aecafe3b1fd608c18d62aa1586d0ab6f88b867bd92b20e0f6bd133606c7c4430`

Funding Information: This study was not funded by any commercial, government or non-profit funding organizations. School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore, India provided the computational facilities and hardware accelerators.

Conflict of Interest: The authors have no competing financial, personal or institutional interests known to them that could have interfered with the work reported in this paper.

Author Contributions: Praveen Kanth N: Conceptualization, CUDA kernel design, micro-architectural optimization of GPU, PCIe streaming pipeline, and manuscript drafting. BharaniKumar J: Modeling, CPU vectorized baseline benchmarking, mathematical formulation and verification test harness for Compressed Sparse Row data. Kishore B: Downstream machine learning pipeline, causal streaming split evaluation, validation thresholding protocol (ADR-007) and visualization of metrics. Dr. Kovendan A. K. P.: Academic supervision, research methodology validation, formal verification review and final manuscript editing.

Open Access Statement: This article is made freely available under the Creative Commons Attribution 4.0 International License (CC BY 4.0), which allows users to use, share, adapt, distribute and reproduce the article in any medium or format, as long as the authors' credit is given.

ACKNOWLEDGMENT

The authors would like to thank the School of Computer Science and Engineering (SCOPE) at Vellore Institute of Technology, Vellore for the experimental infrastructure and also the administrative support.

REFERENCES

[1] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based approaches," Decision Support Systems, vol. 133, p. 113303, 2020.

[2] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2020, pp. 315–324.

[3] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the inconsistency problem of graph neural network-based fraud detectors," in Proc. ACM SIGIR Conf. Res. Dev. Inf. Retrieval, 2020, pp. 569–578.

[4] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," Data Mining and Knowledge Discovery, vol. 29, no. 3, pp. 626–688, 2015.

[5] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q. Z. Sheng, H. Xiong, and L. Akoglu, "A comprehensive survey on graph anomaly detection with deep learning," IEEE Trans. Knowl. Data Eng., vol. 35, no. 12, pp. 12012–12038, 2021.

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017.

[7] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 1024–1034.

[8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.

[9] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

[10] Y. Wang, B. Feng, G. Li, S. Zeng, N. Guan, and X. Shen, "GNNAdvisor: An adaptive and efficient runtime system for GNN acceleration on GPUs," in Proc. USENIX Symp. Oper. Syst. Des. Implementation (OSDI), 2021, pp. 515–531.

[11] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, and D. Fan, "HyGCN: A GCN accelerator with hybrid architecture," in Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA), 2020, pp. 15–29.

[12] Z. Jia, S. Lin, M. Gao, M. Zaharia, and A. Aiken, "Improving the accuracy, scalability, and ease of use of GNN systems with Roc," in Proc. Mach. Learn. Syst. (MLSys), 2020, pp. 187–198.

[13] D. Merrill, M. Garland, and A. Grimshaw, "Scalable GPU graph traversal," in Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP), 2012, pp. 117–128.

[14] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," Science, vol. 286, no. 5439, pp. 509–512, 1999.

[15] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, "Gunrock: A high-performance graph processing library on the GPU," in Proc. ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP), 2016, pp. 1–12.

[16] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in Proc. Conf. High Perform. Comput. Netw. Storage Anal. (SC), 2009, pp. 1–11.

[17] NVIDIA Corporation, "CUDA C++ Programming Guide, Version 12.2," NVIDIA Developer Zone, Santa Clara, CA, Tech. Rep., 2023.

[18] C. Yang, A. Buluç, and J. D. Owens, "Design principles for sparse matrix multiplication on the GPU," in Proc. Euro-Par Int. Conf. Parallel Process., 2018, pp. 672–687.

[19] V. Volkov, "Better performance at lower occupancy," in Proc. GPU Technol. Conf. (GTC), vol. 10, 2010, p. 16.

[20] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, no. 3, pp. 261–272, 2020.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[23] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in Proc. Int. Conf. Mach. Learn. (ICML), 2006, pp. 233–240.

[24] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating imbalanced datasets," PLOS ONE, vol. 10, no. 3, p. e0118432, 2015.